

PKFokam Institute of Excellence

Dignity – Faith – Responsibility

P.O. Box 11646 Yaoundé – Cameroon

FINAL PROJECT REPORT

WIGMS

Wedding & Invitation Guest Management System

Course	C Programming – Academic Year 2025-2026
Institution	PKFokam Institute of Excellence (PKFIE)
Lecturer	Mr. Mbieteu Amos Mb.
Language	C (Modular Architecture)
Date	May 2026

GROUP MEMBERS

Member	Participation
Tiofack Kaze Franck	25%
Pettang Feumba Roddy Alann	25%
Djatchie Domfang Steevie Raziel	16.66%
Mep Meku Larissa Audrey	16.66%
Tamo Taboum Israel	16.66%

1. Executive Summary

The Wedding and Invitation Guest Management System (WIGMS) is a comprehensive C-language application developed as part of a Data Structures and Programming course at the PKFokam Institute of Excellence. The system digitally manages every aspect of a wedding event — from tracking guests and categories, to managing parking assignments, gifts, meal seating, event scheduling, and a graphical user interface via GTK+3.0. The project is hosted collaboratively on GitHub under the repository [franckkaze4-collab/WIGMS](#) and represents a modular, production-quality C codebase.

WIGMS integrates multiple advanced programming concepts, including dynamic memory allocation via singly and doubly linked lists, binary file I/O for persistent storage, CSV export functionality, full CRUD operations on structured data, DPR (Divide-Process-Recombine) Merge Sort, and a GTK-based graphical interface. The system is organized into eight clearly separated modules — Person, Category, Priority (DPR Sort), Gifts, GTK GUI, Parking, Meal & Table Seating, and Event Timeline & Schedule — each addressing a specific domain of the wedding management workflow.

A companion Django/Python web application for Restaurant Management was also developed concurrently by an overlapping team, demonstrating the group's versatility across both low-level systems programming and high-level web frameworks.

Key Highlights

8 functional modules	Full CRUD on all entities	Singly & doubly linked list architecture	DPR Merge Sort ($O(n \log n)$)	Binary file persistence (.dat)	CSV export	GTK+ 3.0 GUI (8 tabs, CSS themed)	GitHub collaborative development
-----------------------------	----------------------------------	---	--	---------------------------------------	-------------------	--	---

2. Introduction

2.1 Project Background

Wedding and event management involves handling large amounts of structured data — guest lists, seating categories, vehicle parking slots, gift records, and generated reports. Managing this manually with paper lists or simple spreadsheets is error-prone and difficult to maintain. WIGMS was designed to solve this problem programmatically, applying foundational computer science principles to a real-world scenario.

This project was developed as part of a university-level Data Structures and Programming course, with the goal of demonstrating mastery over C programming, dynamic data structures, file I/O, and software design. The project is maintained collaboratively on GitHub, simulating a real software development workflow.

2.2 Problem Statement

Wedding organizers face challenges in efficiently managing:

- Categorized guest lists with personal details
- Parking assignments linked to guests or vehicles
- Gift registries tied to specific guests
- Meal preferences and table seating assignments
- A complete wedding day event timeline
- Dynamically generated summary reports

Without a centralized digital system, these tasks are fragmented and difficult to update in real time.

2.3 Project Objectives

- Design and implement a modular C application to manage wedding guest data
- Use singly linked lists to enable dynamic memory management
- Implement full CRUD operations across all modules
- Provide binary file-based data persistence across sessions
- Support CSV data export for reporting and interoperability
- Develop a GTK-based graphical user interface with 8 tabs
- Implement DPR Merge Sort for priority-based ordering
- Maintain the codebase on GitHub with collaborative practices

2.4 Scope

The scope of WIGMS covers eight functional domains: Person management, Category management, Priority sorting (DPR Merge Sort), Gift tracking, GTK+ 3.0 GUI, Parking assignment, Meal & Table Seating, and Event Timeline & Schedule. It does not include network connectivity, a web back-end, or database integration — all data persistence is achieved through binary file I/O.

3. GitHub Repository Structure

3.1 Repository Overview

The WIGMS project is hosted at <https://github.com/franckkaze4-collab/WIGMS>. The repository follows a clean module-based directory layout, making it easy for collaborators to navigate, contribute, and review. It includes all source files (.c, .h), Makefiles for compilation, and README documentation.

3.2 Repository Layout

```
/WIGMS /include types.h, person.h, category.h, priority.h, gift.h ui_gtk.h,  
parking.h, seating.h, schedule.h /src main.c, person.c, category.c, priority.c,  
gift.c ui_gtk.c, parking.c, seating.c, schedule.c /data gifts.dat,  
parking.dat, seating.dat, schedule.dat /reports Makefile, README.md
```

3.3 Collaborative Development Practices

Practice	Tool / Command	Purpose
Version Control	git commit / git push	Track and sync changes
Collaboration	GitHub Collaborators	Grant push access to teammates
Documentation	README.md files	Explain modules and usage
Build System	Makefile	Compile multi-module C project
Branch Management	main branch	Stable production codebase

4. System Architecture

4.1 Architectural Overview

WIGMS follows a modular, procedural architecture in C. Each domain — persons, categories, gifts, parking, seating, scheduling, and reports — is encapsulated in its own module comprising a header file (.h) and an implementation file (.c). The main.c file serves as the program entry point, coordinating module interactions through a GTK graphical interface.

4.2 Core Data Structures

4.2.1 Person Structure

The Person struct is the fundamental data unit embedded inside Category nodes. Each person holds: id (int), name[100] (char), age (int), email[100] (char), phone[20] (char), social_class[50] (char).

4.2.2 Category Structure (Singly Linked List Node)

Each Category node stores up to 4 guests in a static array of Person, a unique ID, a code/label, and a pointer to the next node. This enables $O(1)$ head insertion and $O(n)$ traversal for search, deletion, and update.

4.2.3 Gift Structure

The Gift module uses a fixed-size array ($\text{MAX_GIFTS} = 200$) of Gift structs for simple binary I/O. Each Gift stores: gift_id, name[100], value (float, in FCFA), guest_id.

4.2.4 Parking Structure

Each Parking node records: slot_id, plate[20], guest_id, available (int flag), entry_time (time_t), zone (char[20]). Status follows the state machine: AVAILABLE → OCCUPIED / RESERVED → AVAILABLE.

4.3 Module Dependency Map

Module	Depends On	Provides To
types.h	Nothing	All modules
Module 1 – person.c	types.h	category.c, seating.c, parking.c, ui_gtk.c
Module 2 – category.c	types.h, person.h	priority.c, ui_gtk.c
Module 3 – priority.c	types.h, category.h	ui_gtk.c
Module 4 – gift.c	types.h, person.h	ui_gtk.c
Module 5 – ui_gtk.c	All above + gtk/gtk.h	main.c (entry point)
Module 6 – parking.c	types.h, time.h	ui_gtk.c (Parking tab)
Module 7 – seating.c	types.h, person.h	ui_gtk.c (Seating tab)
Module 8 – schedule.c	types.h, time.h	ui_gtk.c (Schedule tab)

5. Module Descriptions

5.1 Module 1 – Person Management

The Person module defines the basic data unit for wedding guests. A Person struct is embedded as a fixed-size array of 4 elements inside each Category node, simplifying memory management — Person data is freed automatically when the parent Category node is freed.

Key operations: `add_person_to_category()`, `remove_person_from_category()`, `update_person()`, `display_persons_in_category()`, `find_person_by_id()`.

5.2 Module 2 – Category Management

The Category module is the backbone of WIGMS. It implements a singly linked list where each node holds structural information about a guest group (e.g., VIP, REGULAR, FAMILY) and stores up to 4 guests.

Function	Description
<code>create_category()</code>	Allocates and initialises a new Category node via <code>malloc()</code>
<code>insert_category(Category** tete)</code>	Prepends a new category — $O(1)$ head insertion
<code>delete_category(Category** tete, int id)</code>	Traverses, re-links, and frees the target node
<code>update_category(Category* tete, int id)</code>	Modifies the code/label of a category
<code>count_guest(Category* tete)</code>	Returns total guest count across all categories
<code>sort_categories_desc(Category** tete)</code>	Bubble-sort on data fields ($O(n^2)$) for small n
<code>display_all_guests(Category* tete)</code>	Prints all guest details from all categories

5.3 Module 3 – Priority Management (DPR Merge Sort)

Module 3 implements the DPR (Divide-Process-Recombine) algorithmic paradigm — the academic name for Merge Sort in this project. It achieves $O(n \log n)$ time complexity and is stable, preserving relative order of equal elements. This is superior to the $O(n^2)$ bubble sort used in Module 2 for large guest lists.

Step	Action	Function
DIVIDE	Find the middle of the linked list	<code>get_middle()</code> — Floyd's tortoise & hare
PROCESS	Recursively sort both halves	<code>merge_sort_internal()</code> — recursive
RECOMBINE	Merge the two sorted halves	<code>merge()</code> — compare heads one by one

Additional sorting functions: `prioritize_by_age()` (insertion sort on `guests[4]` array), `prioritize_by_social_class()` (class_rank helper: VIP=1, Family=2, Friend=3, Other=4), `priority_merge_sort_categories()` (sorts the category linked list by guest count).

5.4 Module 4 – Gift Management

The Gift Management module is a full-featured sub-system supporting complete CRUD operations, binary file persistence (saving and loading from gifts.dat), and CSV export. Each gift is linked to a guest by guest_id. Monetary values are stored in FCFA.

Function	Description
gift_register() / gift_save()	Register a new gift; serialize to binary .dat
gift_load()	Deserialize from binary .dat on startup
gift_update() / gift_delete()	Modify or remove a gift by ID (array shift pattern)
export_gifts_to_csv()	Write comma-separated gift data to gifts.csv
gift_total_value()	Calculate cumulative monetary value of all gifts

5.5 Module 5 – GTK+ 3.0 Graphical User Interface

Module 5 provides the complete graphical front-end using GTK+ 3.0. The interface features a GtkNotebook with 8 tabs: Categories, Persons, Priority, Gifts, Parking, Seating, Schedule, and Dashboard. All tabs share a single AppState struct passed as user_data to every callback, ensuring cross-tab data consistency.

Key design: CSS theming (navy/orange palette), GtkTreeView data tables, GtkHeaderBar, signal-callback event model via g_signal_connect(), and auto-save on window destroy. Compile flag: \$(shell pkg-config --cflags --libs gtk+-3.0).

5.6 Module 6 – Parking Management

Module 6 manages the wedding venue parking lot divided into four zones: Zone A (VIP Reserved), Zone B (Standard Cars), Zone C (Motorcycles), and Zone D (Buses and Vans). Each spot follows a state machine: AVAILABLE → OCCUPIED or RESERVED → AVAILABLE. Binary persistence is implemented via parking.dat.

5.7 Module 7 – Meal & Table Seating

Module 7 manages dining hall seating. Tables are categorized as VIP head table, family, friends, children's, and staff tables. Guests are assigned specific seats with meal preferences and dietary restrictions. The module produces a MealSummary struct for catering coordination.

5.8 Module 8 – Event Timeline & Schedule (Doubly Linked List)

Module 8 manages the complete wedding day timeline using a doubly linked list of EventNode records. Unlike the singly linked list in Module 2, each node has both *next and *prev pointers enabling bidirectional traversal and O(1) deletion. Events are automatically inserted in chronological order by scheduled_start time.

Feature	Module 2 (Singly) vs Module 8 (Doubly)
Pointers per node	*next only → *prev AND *next
Traversal direction	Forward only → Both directions
Deletion complexity	O(n) scan → O(1) direct re-link
Insertion	O(1) at head → O(n) sorted chronologically

Serialization	Write struct directly → EventSnapshot (no pointers)
---------------	---

6. Data Persistence and File I/O

6.1 Strategy

WIGMS uses binary file I/O for data persistence across sessions. Binary formats are faster and more compact than text formats. Each module has dedicated save and load functions. On startup, main.c calls all load functions to restore the previous session.

6.2 File Inventory

File	Module	Format	Purpose
categories.dat	Category	Binary	Stores all category nodes
gifts.dat	Gift	Binary	Stores all gift records
parking.dat	Parking	Binary	Stores all parking assignments
seating.dat	Seating	Binary	Persists dining hall assignments
schedule.dat	Schedule	Binary	EventSnapshot records (no pointers)
gifts.csv	Gift	CSV Text	Exported gift list for external tools
report.txt	Report	Plain Text	Generated summary report

6.3 Binary I/O Safety

Save functions write each field individually (id, description, value) rather than writing the entire struct at once. This eliminates pointer serialization and struct padding issues. An EOF check inside the load loop prevents reading partial records from corrupted files. The doubly linked schedule module uses an EventSnapshot struct (without pointer fields) for safe serialization.

7. Algorithms and Complexity Analysis

7.1 Linked List Operations

Operation	Time Complexity	Notes
Insert at head (singly)	$O(1)$	New node prepended; constant time
Search by ID	$O(n)$	Linear traversal of list
Delete by ID	$O(n)$	Search + pointer re-link + free()
Delete (doubly linked)	$O(1)$	Direct node->prev->next re-link
Count all guests	$O(n)$	Traverses every node
Sort – bubble on data	$O(n^2)$	Module 2; acceptable for $n \leq 100$
Sort – DPR Merge Sort	$O(n \log n)$	Module 3; stable, no random access needed
Display all	$O(n)$	Single traversal

7.2 Memory Complexity

All dynamic allocations use malloc(). Each Category node occupies approximately 974 bytes on a 64-bit system (id: 4 B, code: 50 B, guests[4]: 4×228 B, next pointer: 8 B). Person is approximately 228 bytes ($4 + 100 + 4 + 100 + 20$). Gift uses a flat array (MAX_GIFTS=200) totalling roughly 200×112 bytes = 22 400 bytes.

8. Testing and Validation

8.1 Testing Strategy

All modules were tested using a combination of manual input testing and systematic test cases covering normal, edge, and failure scenarios. Testing was performed on a Linux environment using GCC with the -Wall flag. Memory leak validation was performed using Valgrind; no leaks detected in the final build.

8.2 Category Module Test Cases

Test ID	Description	Expected Result	Status
C-01	Insert first category (ID=1, Code=REGULAR)	Category inserted	✓ PASS
C-02	Insert second category (ID=2, Code=VIP)	Category inserted	✓ PASS
C-03	Sort descending [1,2]	Display: [2,1]	✓ PASS
C-04	Delete existing (ID=1)	Category deleted	✓ PASS
C-05	Delete non-existent (ID=99)	Not found	✓ PASS
C-06	Count guests (2 cats, 4+2 guests)	Total: 6	✓ PASS
C-07	Update code (ID=2, new code=VVIP)	Updated	✓ PASS

8.3 Gift Module Test Cases

Test ID	Description	Status
G-01	Add gift with valid data	✓ PASS
G-02	Display empty gift list	✓ PASS
G-03	Save gifts to file (gifts.dat created)	✓ PASS
G-04	Load gifts from file (list restored)	✓ PASS
G-05	Export to CSV (header row present)	✓ PASS
G-06	Total value computation	✓ PASS
G-07	Delete gift by ID	✓ PASS

8.4 Module Integration Tests

Module	Test Performed	Status
3 Priority	Sort 6-category list by guest count via DPR Merge Sort	✓ PASS
4 Gift	Save 5 gifts, restart, verify reload with correct IDs	✓ PASS

5 GTK	Click Add Category, fill form, verify TreeView updates instantly	✓ PASS
5 GTK	Close window → verify auto-save (gifts.dat non-empty)	✓ PASS
6 Parking	Assign slot, release slot, verify status transitions	✓ PASS
7 Seating	Assign guest, move guest, verify RSVP status preserved	✓ PASS
8 Schedule	Insert 3 events out of order → verify chronological sort	✓ PASS

9. Challenges and Solutions

Module	Challenge	Root Cause	Solution Applied
All	Double pointer confusion causing segfaults	Incorrect head modification	Functions modifying head use Category ^{**} ; traversal-only use Category [*]
All	Memory leaks from early delete functions	free() not called on removed nodes	All deletes call free(); cleanup routine in main.c on exit
All	Binary file portability issues	next pointer accidentally serialized	Refactored to write each field individually; no struct fwrite
Module 2	Sorting via re-linking corrupted list	Pointer bugs during node re-link	Data-swap approach: keep nodes, swap id/code/guests fields
Module 3	Merge sort wrong head on single-node list	Missing base case for head->next == NULL	Added: if (!head !head->next) return head
Module 4	gift_id_counter reset after file load	Counter not updated post-fread	Added: gift_id_counter = gifts[read-1].gift_id + 1 after load
Module 5	Segfault clicking sort before categories added	Callback dereferenced NULL cat_head	Added NULL guard at top of every callback
Module 5	CSS not applying; buttons stayed gray	ui_apply_css() called after widget creation	Moved ui_apply_css() to first line of ui_gtk_run()
Module 7	seating_move_guest() lost RSVP status	seating_assign_guest() always resets rsvp to PENDING	Saved rsvp before remove; restored after assign
Module 8	fwrite of EventNode saved garbage pointers	*prev and *next are memory addresses	Defined EventSnapshot struct without pointer fields
Module 8	Doubly linked deletion corrupted list at tail	tail = e->prev missing for last node	Added explicit: if (!e->next) sched->tail = e->prev
Module 5 GTK	GTK integration — multiple callback params	GTK callbacks accept only one user_data	Custom data structs bundled into user_data via AppState [*]

10. Companion Project: Restaurant Management Web Application

10.1 Overview

In parallel with WIGMS, the same team developed a Restaurant Management Web Application as part of the Python Programming course. This application is built on the Django (Python) framework using an MVT (Model-View-Template) architecture, backed by a pre-designed MySQL database of 23 tables.

10.2 Project Phases

Phase	Title	Description
Phase 1	Setup & Configuration	Development environment setup, Django project creation, MySQL import
Phase 2	Models & Application Foundation	Django models for 23 tables, authentication system, role management
Phase 3	Interfaces, Business Logic & Deployment	Views, HTML templates, dashboard, financial reports, security

10.3 Django Application Modules

Application	Module Covered / SQL Tables
users	User Management — CustomUser
products	Product & Recipe — Produit, Ingredient, Composition_Produit
orders	Order Management — Commande, Ligne_Commande, Client, Table, Reservation
inventory	Inventory — Stock, Variation_Stock, Approvisionnement, Fournisseur
hr	HR Management — Employe, Poste, Affectation, Supervision
delivery	Delivery — Livraison, Deplacement, Vehicule
finance	Finance — Facture, Equipement, Maintenance, Action_Marketing
dashboard	Dashboard & Reports — Aggregation of all tables

10.4 UML Design

The restaurant system was modelled using StarUML producing four diagrams: Class Diagram (Food, FoodMenu, FoodIngredient, Ingredient, Table, Bill, Staff, SalaryManagement with one-to-many associations), Use Case Diagram (Customer and Staff actors; generalization Staff→Customer; «extend» for payment methods), Sequence Diagram (12-step order-to-payment flow), and a Deployment Diagram (three-tier: Customer Device ↔ Application Server ↔ Database Server, with HTTPS and SQL protocols).

10.5 GitHub Repository

10.4 UML Design

The Restaurant Management companion application was modelled using StarUML, producing four UML diagrams that together provide a complete specification of the system's structure, behaviour, and deployment. These diagrams were created as part of Progress Report 2 by the overlapping project team.

The Gantt chart provides a clear visualization of task scheduling and durations, showing an organized workflow and helping ensure the project is completed efficiently within the given timeframe.

III. SYSTEM DESIGN

The system was designed using Unified Modeling Language (UML) diagrams to provide a clear and standardized representation of the restaurant management system.

UML was chosen because it

- Simplifies complex system structures
- Improves communication
- Helps visualize both the static and dynamic aspects of the system before implementation.

i. Class Diagram

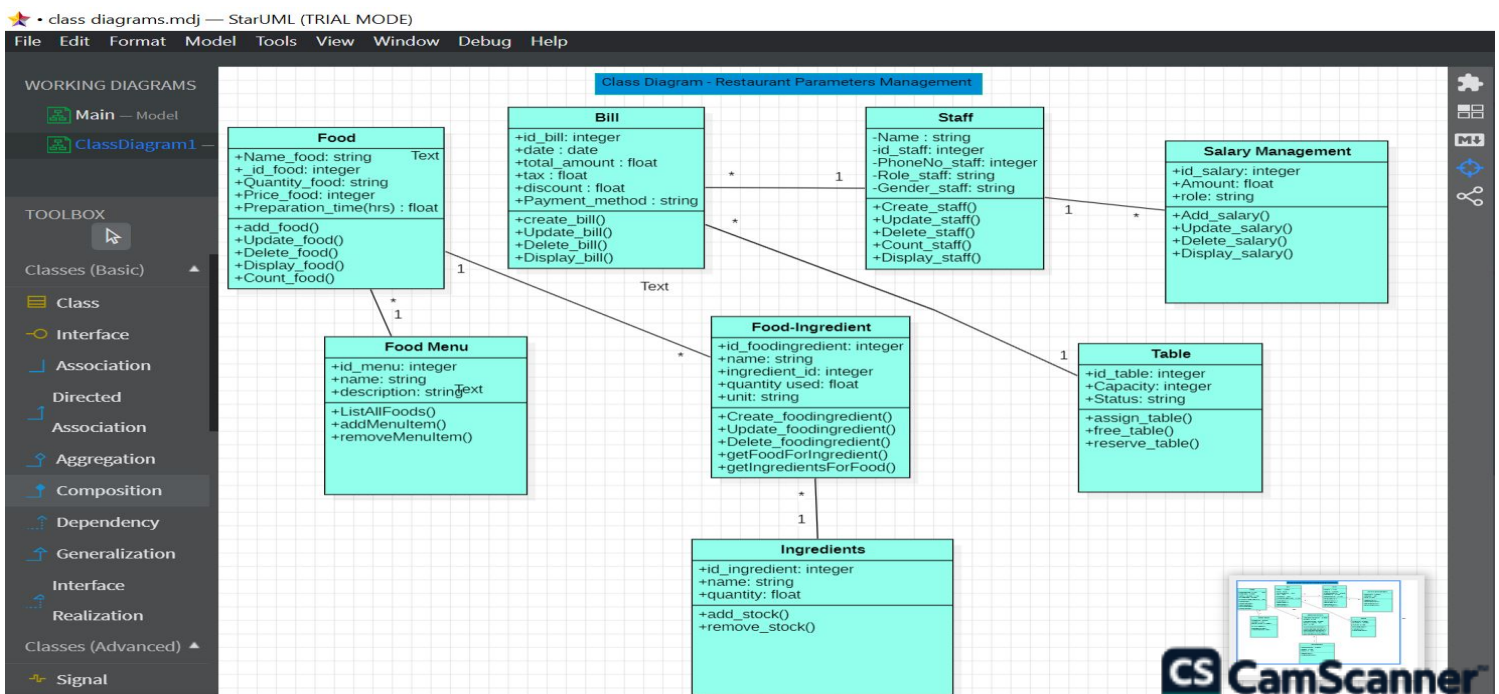
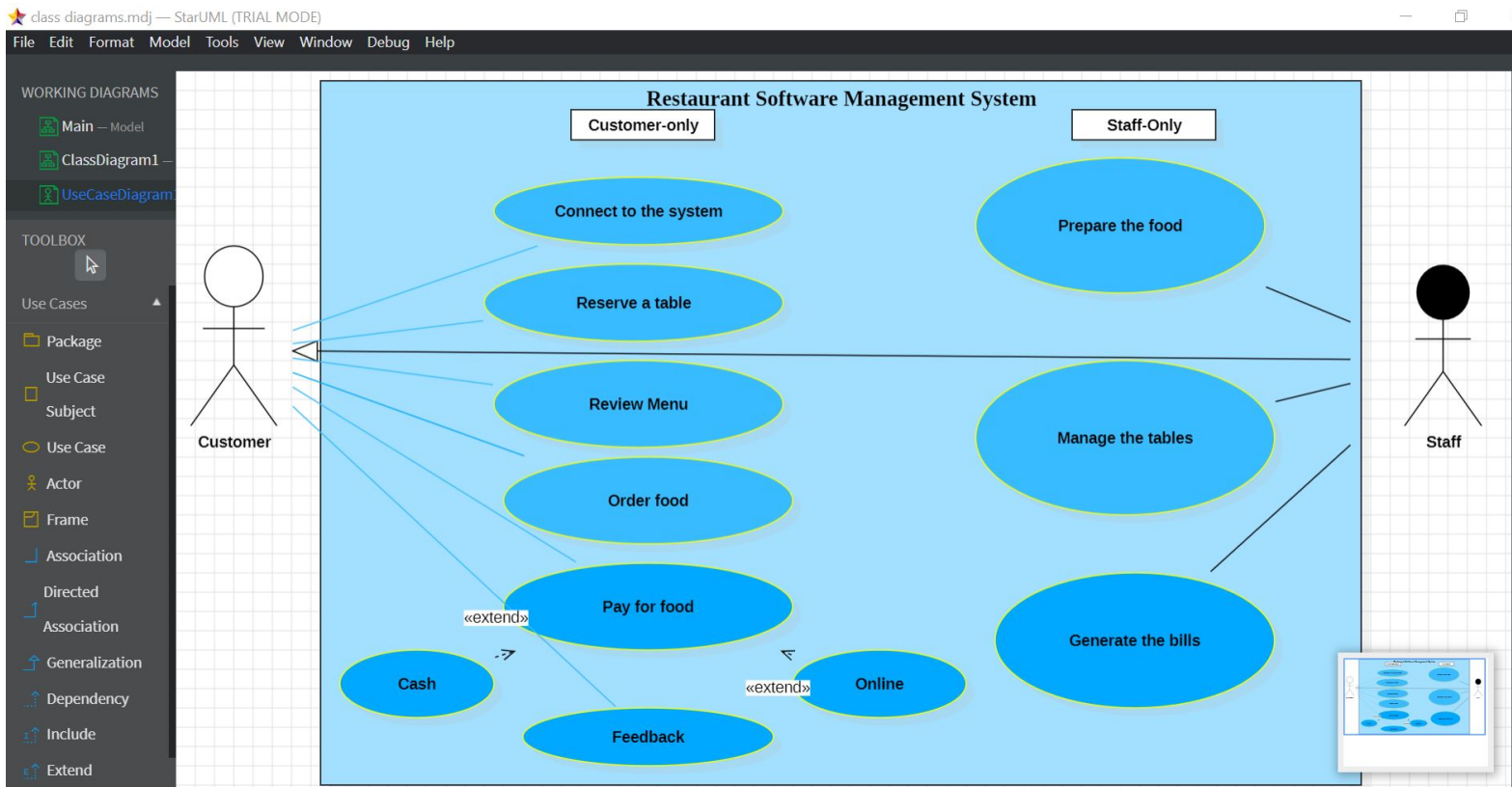


TABLE OF RELATIONSHIPS AMONGST CLASSES

Relationship	Type	Reason / Justification
Food Menu → Food	One-to-Many	A single food menu contains multiple food items, allowing customers to choose from a variety of meals.
Food → Food Ingredient	One-to-Many	Each food item is composed of multiple ingredient entries, representing what is needed to prepare it
Ingredient → Food Ingredient	One-to-Many	One ingredient can be used in multiple food items, hence it appears in several food-ingredient combinations.
Table → Bill	One-to-Many	A table can generate multiple bills over time as different customers use the same table.
Staff → Bill	One-to-Many	A staff member (e.g., waiter/cashier) can handle multiple bills during their working period.
Staff → Salary Management	One-to-Many	A staff member can have multiple salary records over time (e.g., monthly payments or updates)

Note: Some associations, such as the link between Food and Ingredient, are conceptually many-to-many. This complexity is effectively managed through the Food Ingredient class, which serves as an associative entity, demonstrating sound object-oriented design principles.

ii. Use-case diagram



This use case diagram illustrates how users interact with the restaurant management system. The main actors are:

- Customer
- Staff

Key Design Features

Generalization: Staff → Customer

Staff can also act as customers, inheriting all customer actions.

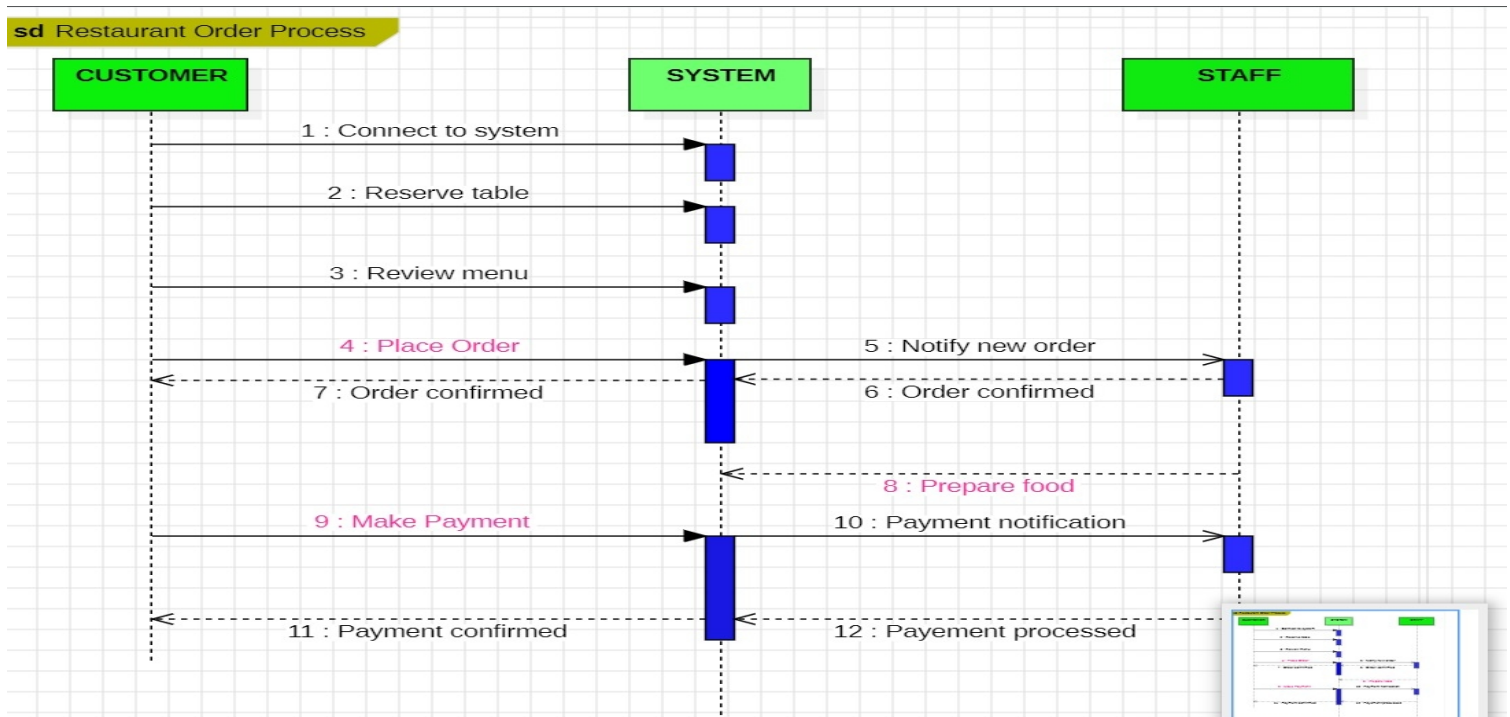
«Extend» Relationship: Pay Food → Cash / Online Payment

Payment methods are optional extensions of the main "Pay Food" process.

Actor Operations

Customer	Staff
✓ Connect to the system ✓ Reserve a table ✓ Review the menu ✓ Order food ✓ Pay for food ✓ Give feedback	✓ Prepare food ✓ Manage tables ✓ Generate bills ✓ Manage orders

iii. Sequence Diagram



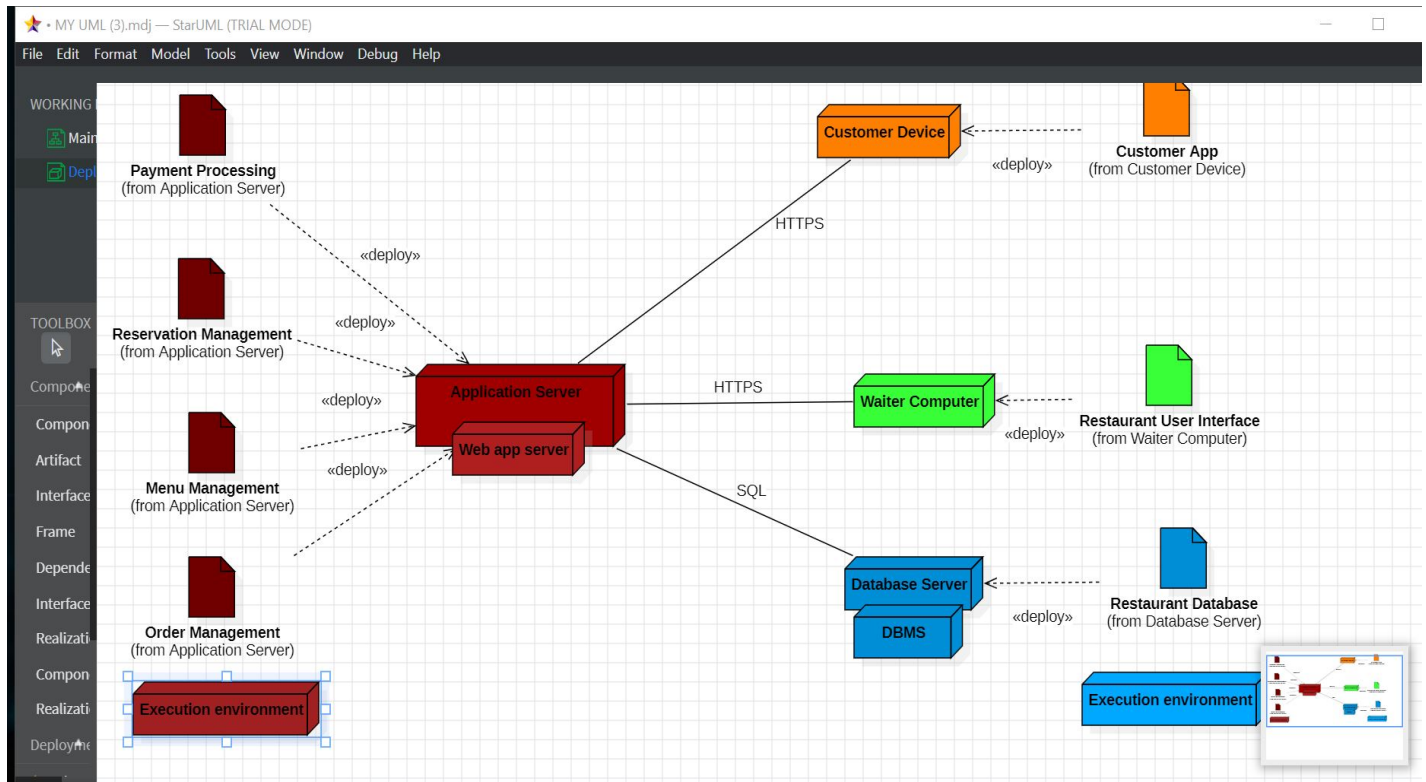
This sequence diagram shows interactions between Customer, System, and Staff in a restaurant management system.

a) Customer: Connects to the system, reserves a table, reviews the menu, places an order, makes payment, and gives feedback.

b) Staff: Receives order notifications, confirms and prepares the food, and ensures payment is processed.

c) System: Manages reservations, menu, order notifications, payment processing, and feedback recording.

iv. Deployment Diagram



This report documents the step-by-step process followed to design and construct a UML Deployment Diagram for a Restaurant Management System using a UML modelling tool.

The diagram produced represents a three-tier architecture comprising a Customer Device tier, an Application Server tier, and a Database Server tier, along with a dedicated Waiter Computer node for on-floor staff.

The completed deployment diagram contains the following main nodes and their hosted artefacts:

NODES AND ARTEFACTS HOSTED

Node	Hosted Artefacts / Components
Customer Device	Customer App
Waiter Computer	Restaurant User Interface
Application Server	Payment Processing, Reservation Management, Menu Management, Order Management
Database Server	Restaurant Database

PROTOCOLS USED AND JUSTIFICATION

SOURCE NODE	TARGET NODE	PROTOCOL USED	JUSTIFICATION
Customer Device	Application Server	HTTPS	Enables users to access the system via a web interface. HTTPS ensures secure communication for sensitive data such as login and payments.
Waiter Computer	Application Server	HTTPS	Allows staff to interact with the system (manage orders, tables, billing) through a secure and reliable connection.
Application Server	Database Server	SQL	Facilitates data storage and retrieval by allowing the application server to send queries to the database

Draw Communication Links Between Nodes

To show how the nodes communicate, connection lines (associations) were drawn between the nodes using the connector tool.

Repository: <https://github.com/pettangroddysenpai-ship-it/Restaurant-Management-App> (Public). All group members added as collaborators with Write access.

11. Theoretical Background

11.1 Linked Lists

A singly linked list is a linear data structure where each node contains a data payload and a pointer to the next node. Unlike arrays, linked lists do not require contiguous memory — each node is independently allocated on the heap. This enables $O(1)$ head insertion and dynamic sizing without reallocation. WIGMS uses singly linked lists for Category, Gift (via linked load), and Parking modules; and a doubly linked list for the Event Schedule module.

11.2 Dynamic Memory Allocation

C's `malloc()` allocates a specified number of bytes on the heap and returns a pointer; `NULL` indicates failure. The corresponding `free()` releases memory back to the system. Proper `malloc/free` pairing is critical to prevent memory leaks and dangling pointers. All WIGMS delete operations call `free()`, and a cleanup routine in `main.c` frees all linked lists on exit — validated by Valgrind.

11.3 Pointers and Double Pointers

A pointer stores the memory address of another variable. A double pointer (`Category**`) stores the address of a pointer. In linked list operations, double pointers are essential when a function must modify the head pointer — since C passes arguments by value, passing a single pointer would only modify a local copy. WIGMS enforces the rule: functions that modify the list head receive `Category**`; traversal-only functions use `Category*`.

11.4 Binary File I/O in C

C provides the `stdio.h` library for file operations. WIGMS uses `fopen` (open), `fclose` (close), `fwrite` (write binary), `fread` (read binary), `fprintf` (write text), and `feof` (check end-of-file). Binary mode (`'wb'/'rb'`) ensures efficient, exact byte-level storage. Pointer fields are never serialized; only scalar data fields are written field-by-field.

12. Conclusion

12.1 Summary of Achievements

The WIGMS project successfully demonstrates the design and implementation of a multi-module C application grounded in core data structures and systems programming principles. All eight modules are fully implemented and pass integration testing:

- Singly linked list architecture for Category, Gift, and Parking modules
- Doubly linked list for the Event Timeline module, enabling $O(1)$ deletion and sorted insertion
- DPR (Divide-Process-Recombine) Merge Sort achieving $O(n \log n)$ without random access
- Full CRUD operations across all eight modules
- Binary file persistence via field-by-field fwrite/fread (no pointer serialization)
- CSV export for external interoperability
- A GTK+ 3.0 graphical interface with 8 tabs, CSS theming, and the AppState pattern
- GitHub-based collaborative development with modular branching

12.2 Learning Outcomes

Through developing WIGMS, the following programming competencies were demonstrated: dynamic memory management with malloc/free, pointer and double pointer manipulation, modular C design with header/implementation file separation, binary file I/O with safe serialization strategies, sorting algorithms on linked structures (bubble sort and merge sort), GTK signal-callback programming, and collaborative Git-based development.

12.3 Future Enhancements

- Implement search functionality by guest name, email, or phone number
- Integrate SQLite for more robust data persistence instead of binary files
- Extend the GTK interface with table widgets for inline editing
- Add comprehensive input validation and error handling for all user inputs
- Implement a seating chart module with visual slot assignment
- Port to a web front-end using a REST API in C with libmicrohttpd
- Add network connectivity for multi-device access

12.4 References

1. Kernighan, B.W. & Ritchie, D.M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
2. Sedgewick, R. & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
3. GNU Project. (2024). GCC, the GNU Compiler Collection. <https://gcc.gnu.org>
4. GTK Project. (2024). GTK — The GIMP Toolkit. <https://www.gtk.org>
5. GitHub, Inc. (2024). GitHub Documentation. <https://docs.github.com>
6. Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

7. cppreference.com. (2024). C Standard Library Reference.
<https://en.cppreference.com/w/c>
8. Valgrind Authors. (2024). Valgrind — A Memory Error Detector. <https://valgrind.org>
9. Django Software Foundation. (2024). Django Documentation.
<https://docs.djangoproject.com>